Project BlueLine — Complete Project Documentation

Version: 1.0 **Date:** April 2026 **Prepared by:** LTM AI-Led Engineering Team **Repository:** https://github.com/sagu25/blueline.git **Status:** POC Delivered | Production Architecture Designed

---

Table of Contents

1. Project Overview
2. System Architecture
3. The Three Automation Tracks
4. All 12 Agents — Complete Reference
5. Technology Stack
6. Project Folder Structure
7. Local Setup & Installation
8. Configuration & Environment Variables
9. Running the POC Application
10. System Integrations
11. Production Architecture
12. Human Control Points & Safety Design
13. Shadow Mode
14. Testing Guide
15. Deployment Guide
16. DAS Coding Standards Reference
17. Glossary

---

1. Project Overview

What Is Project BlueLine?

Project BlueLine is an **AI-powered engineering automation system** that replaces three heavily manual workflows with intelligent agents running automatically in the background on Azure infrastructure.

It targets engineering teams working on **.NET (C#) and Angular (TypeScript)** codebases under **Azure DevOps**, integrating with **Fortify SSC** for security scanning and **Azure Key Vault** for certificate management.

The Three Problems It Solves

#	Problem	Current Pain	BlueLine Solution
1	**Code Review**	Every PR manually checked — slow, inconsistent, reviewer-dependent	AI agents review every PR on open, post inline comments, flag risks within 45 seconds
2	**Security (Fortify)**	Fortify produces finding lists — engineers manually triage, research, and fix each one	Agents triage findings with AI, classify by severity, and generate fix code as draft PRs
3	**SSL Certificates**	Tracked via spreadsheet — manual requests, installs across Dev/QA/Prod	Agents detect expiry, raise renewal, and deploy to all environments automatically

Core Design Principle

> ***Humans stay in control.*** *Agents analyse and prepare. Humans decide and approve. No code is merged and no Production deployment happens without a human approving it.*

Project Status

Track	POC Status	Production Status
Quality Gate (Code Review)	**Fully built and runnable**	Architecture designed; ready to deploy
Security Loop (Fortify)	Core agents built; Fortify connection simulated	Awaiting Fortify SSC API access
Certificate Loop	Core agents built; CA and IIS simulated	Awaiting Key Vault + WinRM access

---

2. System Architecture

High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          EXTERNAL TRIGGERS                               │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐       │
│  │  Azure DevOps    │  │   Fortify SSC    │  │   Azure Timer    │       │
│  │  PR Webhook      │  │  Pipeline Hook   │  │   (Daily)        │       │
│  └──────────┬───────┘  └──────────┬───────┘  └──────────┬───────┘       │
└─────────────┼──────────────────────┼──────────────────────┼──────────────┘
              │                      │                      │
┌─────────────▼──────────────────────▼──────────────────────▼──────────────┐
│                   AZURE API MANAGEMENT (Gateway)                          │
│              Auth · Rate Limiting · Request Routing                       │
└─────────────┼──────────────────────┼──────────────────────┼──────────────┘
              │                      │                      │
┌─────────────▼────────┐  ┌──────────▼──────────┐  ┌────────▼────────────┐
│  QUALITY GATE        │  │   SECURITY LOOP     │  │  CERTIFICATE LOOP   │
│  Function App        │  │   Function App      │  │  Function App       │
│                      │  │                     │  │                     │
│  CLARION             │  │  WATCHTOWER         │  │  TIMELINE           │
│  LUMEN               │  │  BULWARK            │  │  REGENT             │
│  VECTOR              │  │  FORGE              │  │  COURIER            │
│  ASCENT              │  │  STEWARD            │  │  HARBOUR            │
└──────────┬───────────┘  └──────────┬──────────┘  └──────────┬──────────┘
           │                         │                        │
┌──────────▼─────────────────────────▼────────────────────────▼──────────┐
│                        SHARED SERVICES LAYER                            │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐         │
│  │  Claude API  │  │ Azure Service │  │  Azure Monitor +     │         │
│  │  (LLM Core)  │  │ Bus (Messaging)│  │  Log Analytics       │         │
│  └──────────────┘  └──────────────┘  └──────────────────────┘         │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐         │
│  │  Azure Key   │  │ Azure Storage │  │  Azure Table Storage │         │
│  │  Vault       │  │ (Audit Logs)  │  │  (Cert Inventory)    │         │
│  └──────────────┘  └──────────────┘  └──────────────────────┘         │
└──────────────────────────────────────────────────────────────────────────┘
           │                         │                        │
┌──────────▼──────────┐  ┌──────────▼──────────┐  ┌──────────▼──────────┐
│  Azure DevOps       │  │   Fortify SSC       │  │  Azure Key Vault /  │
│  / GitHub API       │  │   REST API          │  │  IIS / App Service  │
└─────────────────────┘  └─────────────────────┘  └─────────────────────┘
```

How Agents Communicate

Agents do **not** call each other directly. They communicate through **Azure Service Bus** — a message queue. One agent publishes a message, the next picks it up independently.

```
WATCHTOWER  ──publishes──► "security.findings.new"  ──► BULWARK
BULWARK     ──publishes──► "security.critical.fix-needed" ──► FORGE
ALL agents  ──publish───► all events  ──► STEWARD (subscribes to all)
```

Why this matters:

- Agents are independent — if FORGE is slow, BULWARK is not blocked
- Agents are replaceable — swap one without touching others
- Messages are persisted — if an agent crashes, the message is not lost

Quality Gate — Parallel Execution

The Quality Gate uses **Azure Durable Functions** to run CLARION, LUMEN, and VECTOR simultaneously, then waits for all three before ASCENT posts its summary:

```
PR arrives
    │
    ├── CLARION ──┐
    ├── LUMEN     ├── all three run at the same time (~30-45 seconds total)
    └── VECTOR ───┘
                │
                ▼  (all three done)
            ASCENT
        aggregates results
        posts one consolidated comment to the PR
```

---

3. The Three Automation Tracks

Track 1 — Quality Gate (Code Review)

Trigger: Pull Request opened or updated in Azure DevOps

Flow:

```
Developer opens PR in Azure DevOps
         │
         ▼
Webhook fires → BlueLine receives PR ID
         │
    ┌────┴──────────────┐
    ▼        ▼        ▼
 CLARION    LUMEN     VECTOR
 (standards)(smells)  (risk)
    │        │        │
    └────────┴────────┘
         ▼
       ASCENT
  aggregates + posts
  one summary comment:
  ┌──────────────────────────┐
  │ RECOMMENDATION: BLOCK    │
  │ Must Fix: 2  |  Warn: 3  │
  │ Risk Score: 8.4/10       │
  │ Reviewer must check: [X] │
  └──────────────────────────┘
         │
         ▼
Human reviewer focuses on logic
Human approves or requests changes
(agents cannot merge — ever)
```

Output per PR:

- Inline comment on each violation (file + line + rule + fix)
- One consolidated ASCENT summary comment
- Recommendation: APPROVE / REQUEST CHANGES / BLOCK
- PR risk score (0–10)
- Tiered finding list (Must Fix / Should Fix / Consider Fixing)

Track 2 — Security Loop (Fortify)

Trigger: CI/CD pipeline run or WATCHTOWER scheduled scan

Flow:

```
Fortify scan completes (pipeline or schedule)
         |
         ▼
WATCHTOWER detects new findings
         |
         ▼
BULWARK triages each finding:
┌──────────────────────────────────────┐
│  SQL Injection        → CRITICAL     │
│  Unused variable      → FALSE_POSITIVE │
│  Insecure config      → HIGH         │
│  Ambiguous pattern    → NEEDS_REVIEW  │
└──────────────────────────────────────┘
         |
  CRITICAL/HIGH ──────────────────────┐
         |                            |
         ▼                            ▼
FORGE reads source file          STEWARD logs everything
Generates fix code               to immutable audit trail
Creates draft fix PR:
┌──────────────────────────────────────┐
│  Branch: fix/sql-injection-line-142  │
│  Fix: parameterized query applied    │
│  Test: unit test added               │
│  Status: DRAFT — awaiting approval   │
└──────────────────────────────────────┘
         |
         ▼
Human reviews + approves fix PR
(agents cannot merge — ever)
```

Track 3 — Certificate Loop (SSL Renewal)

Trigger: Azure Timer (daily schedule)

Flow:

```
Every morning — TIMELINE runs
         |
         ▼
Queries Azure Key Vault for all certs
Flags expiring within 30 days:
┌──────────────────────────────────────┐
│  api.example.com      → 12 days left     │
│  payments.portal.com  → 28 days left     │
└──────────────────────────────────────┘
         |
         ▼
REGENT updates the certificate inventory
         |
         ▼
COURIER raises renewal with CA:
Internal → C&M portal API
External → DigiCert/InfoSec API
         |
         ▼
Downloads + validates renewed cert:
  expiry ✅  domain ✅  chain ✅
         |
         ▼
HARBOUR deploys automatically:
Dev  ──install──verify HTTPS ✅
QA   ──install──verify HTTPS ✅
Prod ──HOLDS──────────────────────────┐
                                      |
Teams card sent to approver:          |
┌──────────────────────────────────┐  |
│  Cert ready for Production          |  |
│  Domain: api.example.com            |  |
│  New expiry: 2027-04-27            |  |
│  [APPROVE]  [REJECT]                |  |
└──────────────────────────────────┘  |
         |                            |
Human clicks APPROVE ◄────────────────┘
         |
HARBOUR deploys to Prod
Verifies HTTPS ✅
STEWARD writes audit log
```

---

4. All 12 Agents — Complete Reference

4.1 CLARION — Coding Standards Checker

Track: Quality Gate **File:** `poc/agents/clarion.py` **Trigger:** PR opened or updated

What it does: Checks every changed file against the team's DAS/CDAS coding standards for .NET (C#) and Angular (TypeScript).

Standards enforced:

- C# naming conventions (PascalCase for types/methods, camelCase for params, `I` prefix for interfaces)
- Critical async patterns — no `.Result` or `.Wait()`, always use `ConfigureAwait(false)`, `CancellationToken` propagation
- HttpClient management — use `IHttpClientFactory`, never `new HttpClient()` directly
- Secrets management — no hardcoded connection strings, keys, or passwords (use Key Vault)
- Exception handling — never expose raw error messages to API callers
- Input validation — `ModelState.IsValid`, FluentValidation, no raw user input passed to queries
- CORS security — no wildcard origins with credentials
- Dependency injection patterns — constructor injection only
- Entity Framework best practices — `AsNoTracking()` for read-only, no N+1 queries
- Logging standards — structured logging with Serilog/ILogger
- Angular/TypeScript — `OnPush` change detection, typed HTTP clients, no `any` type

Input: Code snippet (C# or TypeScript) **Output:**

```
{
  "violations": [
    {
      "rule": "ASYNC_001",
      "severity": "error",
      "line": 42,
      "message": "Using .Result blocks the thread — use await instead",
      "fix": "await GetDataAsync()",
      "confidence": 0.95
    }
  ],
  "files_checked": 1,
  "language": "csharp"
}
```

4.2 LUMEN — Code Smell & Anti-Pattern Detector

Track: Quality Gate **File:** `poc/agents/lumen.py` **Trigger:** PR opened or updated

What it does: Detects code quality and maintainability issues that are not strict rule violations but will cause problems over time.

Smells detected:

- Long methods (>40 lines)

- Large classes (>300 lines)
- Deep nesting (>3 levels)
- Magic numbers and strings (unnamed literals)
- Duplicate code blocks
- Dead code (unreachable or unused)
- Long parameter lists (>4 parameters)
- Feature envy (method uses another class's data more than its own)
- God classes (class doing too much)
- Primitive obsession (using primitives instead of domain types)
- .NET specific: `DbContext` as singleton, blocking async calls, N+1 queries

Input: Code snippet **Output:**

```
{
  "smells": [
    {
      "type": "LONG_METHOD",
      "severity": "major",
      "location": "ProcessOrderAsync (line 15–87)",
      "explanation": "72-line method doing 5 distinct things",
      "refactoring_suggestion": "Extract into ValidateOrder(), CalculateTax(), SaveOrder(), NotifyCustomer()",
      "effort": "medium"
    }
  ],
  "maintainability_score": 4,
  "summary": "3 major smells found"
}
```

4.3 VECTOR — Risk & Complexity Scorer

Track: Quality Gate **File:** `poc/agents/vector.py` **Trigger:** PR opened or updated

What it does: Scores each file by risk level and identifies hotspots — tells the human reviewer *where to look* in the PR.

Metrics computed locally (before LLM call):

- Cyclomatic complexity (count of decision branches)
- Maximum nesting depth (indentation-based)
- Method/function count
- Import/dependency count
- Security operation detection (SQL, HTTP calls, auth, crypto patterns)
- Test code presence detection

- Blocking async calls (`.Result`, `.Wait`)
- High-risk patterns (`new HttpClient`, file uploads, CORS wildcards)
- Empty catch blocks

Input: Code snippet **Output:**

```
{
  "overall_risk_score": 0.82,
  "risk_level": "HIGH",
  "metrics": {
    "cyclomatic_complexity": 14,
    "max_nesting_depth": 5,
    "method_count": 8,
    "has_security_operations": true
  },
  "hotspots": [
    {
      "location": "line 34–67",
      "reason": "Complex auth logic with 5 nested conditions",
      "reviewer_focus": "Verify all auth failure paths return 401, not 500"
    }
  ],
  "reviewer_attention": "Focus on: auth flow complexity and SQL query parameters"
}
```

4.4 ASCENT — Aggregator & Final Recommendation

Track: Quality Gate **File:** `poc/agents/ascent.py` **Trigger:** After CLARION, LUMEN, and VECTOR have all completed

What it does: Reads all three agents' outputs and produces one consolidated PR review comment with a clear recommendation.

Recommendation logic:

- `BLOCK` — any error-severity violation or confirmed security issue
- `REQUEST_CHANGES` — error-level violations or significant warnings
- `APPROVE` — minor issues only, no blockers

Tiers:

- **Tier 1 — Must Fix Before Merge:** Errors, security issues, CRITICAL risk findings
- **Tier 2 — Should Fix:** Warnings, major smells, HIGH risk findings
- **Tier 3 — Consider Fixing:** Info-level, minor smells, MEDIUM risk

Input: Combined JSON from CLARION + LUMEN + VECTOR **Output (posted as PR comment):**

```
## BlueLine Review — REQUEST CHANGES

**Overall Score: 5.2/10** | Risk: HIGH

### Must Fix Before Merge (2 issues)
- [CLARION] Line 42: Using .Result blocks thread — use await
- [VECTOR] Auth logic complexity critical — verify all failure paths

### Should Fix (3 issues)
- [LUMEN] ProcessOrderAsync is 72 lines — extract to smaller methods
...

### Reviewer Checklist
- [ ] Verify auth flow returns correct status codes
- [ ] Check SQL parameters are all parameterized

_Reviewed by BlueLine ASCENT | Score: 5.2/10 | 43 seconds_
```

4.5 WATCHTOWER — Fortify Scan Monitor

Track: Security Loop **File:** `poc/agents/watchtower.py` **Trigger:** CI/CD pipeline hook or scheduled timer

What it does: Monitors Fortify SSC for new vulnerability findings. When new findings appear, publishes them to the security pipeline for BULWARK to triage.

Production integration:

- Polls Fortify SSC REST API
- Filters findings by project, date, and status
- Publishes to Azure Service Bus topic: `security.findings.new`

POC: Finding input is simulated via the Streamlit UI (paste a finding description)

4.6 BULWARK — Security Finding Triage Agent

Track: Security Loop **File:** `poc/agents/bulwark.py` **Trigger:** New finding from WATCHTOWER

What it does: Classifies each Fortify finding using AI and OWASP Top 10 knowledge. Tells engineers whether to fix it now, review it, or mark it as a false positive — with the reasoning.

Classifications: | Label | Meaning | |---|---| | `CRITICAL` | Confirmed, exploitable, fix immediately | | `HIGH` | Very likely vulnerable, high priority | | `NEEDS_REVIEW` | Possible vulnerability, needs human verification | | `FALSE_POSITIVE` | Not actually vulnerable — reasoning provided |

Input: Fortify finding description + optional code snippet **Output:**

```
{
  "classification": "CRITICAL",
  "confidence": 0.94,
  "owasp_category": "A03:2021 – Injection",
  "attack_scenario": "Attacker can inject SQL via the 'username' parameter and read all user records",
  "affected_systems": "User authentication database",
  "secure_code_example": "cmd.Parameters.AddWithValue(\"@username\", username);",
  "false_positive_reason": null
}
```

4.7 FORGE — Security Fix PR Creator

Track: Security Loop **File:** `poc/agents/forge.py` **Trigger:** CRITICAL or HIGH classification from BULWARK

What it does: For every critical/high security finding, generates the actual fix code and creates a draft Pull Request in Azure DevOps. The PR is always in DRAFT state — it cannot be merged without human approval.

Input: Finding description + BULWARK classification + vulnerable code **Output (draft PR metadata):**

```
{
  "branch_name": "fix/security/sql-injection-usercontroller-line-142",
  "commit_message": "fix(security): parameterize SQL query in UserController to prevent injection",
  "pr_title": "[SECURITY FIX] SQL Injection — UserController.cs:142",
  "pr_description": "## Security Fix\n**Finding:** SQL Injection...\n**Fix:** Replaced string concatenation with
  "files_to_modify": ["Controllers/UserController.cs"],
  "ready_to_merge": false,
  "reviewer_note": "Verify all SQL queries in this controller are similarly parameterized"
}
```

4.8 STEWARD — Audit Log Agent

Track: Security Loop **File:** `poc/agents/steward.py` **Trigger:** All security pipeline events

What it does: Creates an immutable, structured audit log entry for every action taken in the security pipeline. Every finding, every classification, every fix, every suppression — all logged with reasoning and timestamp.

Production storage: Azure Blob Storage with WORM (Write Once Read Many) policy and 7-year retention.

Output:

```
{
  "run_id": "STEWARD-20260427-143502-A3F9",
  "timestamp_utc": "2026-04-27T14:35:02Z",
  "pipeline": "security_loop",
  "finding_summary": "SQL Injection in UserController.cs:142",
  "classification": "CRITICAL",
  "confidence": 0.94,
  "action_taken": "FORGE draft PR created: fix/security/sql-injection-usercontroller-line-142",
  "human_gate_required": true,
  "human_gate_status": "PENDING",
  "immutable": true,
  "retention_policy": "7_years"
}
```

4.9 TIMELINE — Certificate Expiry Monitor

Track: Certificate Loop **File:** `poc/agents/timeline.py` **Trigger:** Daily Azure Timer

What it does: Queries Azure Key Vault for all certificates and flags any expiring within threshold windows.

Urgency levels: | Level | Condition | |---|---| | `EXPIRED` | Already expired | | `CRITICAL` | < 7 days remaining | | `URGENT` | < 14 days remaining | | `RENEWAL_NEEDED` | < 30 days remaining | | `MONITOR` | 30–90 days remaining | | `OK` | > 90 days remaining |

Input: Certificate metadata (subject, expiry date, environments, CA type) **Output:**

```
{
  "urgency": "URGENT",
  "days_until_expiry": 11,
  "risk_level": "HIGH",
  "renewal_path": "internal_pki",
  "action_plan": [
    "1. Generate CSR via COURIER",
    "2. Submit to C&M portal",
    "3. Download issued cert",
    "4. Deploy via HARBOUR to Dev → QA → Prod"
  ],
  "automation_possible": true,
  "risks": ["IIS binding may need manual update if thumbprint changes"]
}
```

4.10 REGENT — Certificate Inventory Manager

Track: Certificate Loop **File:** `poc/agents/regent.py` **Trigger:** TIMELINE output

What it does: Maintains a structured inventory of all SSL/TLS certificates across all environments. Tracks owner, CA type, environments, expiry, and status.

POC: In-memory sample database with 4 certificates. **Production:** Azure Table Storage — updated by all certificate agents.

Sample inventory entry:

```
{
  "name": "api.coreandmain.com",
  "subject": "api.coreandmain.com",
  "owner": "platform-team",
  "ca_type": "internal",
  "environments": ["dev", "qa", "prod"],
  "expiry_date": "2026-05-08",
  "days_remaining": 11,
  "status": "URGENT",
  "deployment_targets": {
    "dev": "IIS — devserver01",
    "qa": "IIS — qaserver01",
    "prod": "Azure App Service — api-prod"
  }
}
```

4.11 COURIER — Certificate Renewal Requester

Track: Certificate Loop **File:** `poc/agents/courier.py` **Trigger:** TIMELINE urgency flag

What it does: Calls the appropriate Certificate Authority API to request a certificate renewal and downloads the issued certificate.

Supported CAs:

- Internal PKI (C&M portal API)
- DigiCert (external CA API)
- Let's Encrypt (ACME protocol)

Input: Certificate subject, CA type, environments, days remaining **Output:**

```
{
  "request_summary": "Renewal request submitted to Internal PKI",
  "ca_order_id": "PKI-2026-0427-00142",
  "validation_method": "Internal auto-approval (no manual step needed)",
  "estimated_delivery": "2–4 hours",
  "cert_download_ready": true,
  "cert_format": "PFX",
  "simulated_thumbprint": "3A:9F:...:B2",
  "next_steps_for_harbour": "Deploy PFX to IIS; bind to site; verify HTTPS"
}
```

4.12 HARBOUR — Certificate Deployment Agent

Track: Certificate Loop **File:** `poc/agents/harbour.py` **Trigger:** COURIER output (renewed certificate ready)

What it does: Installs the renewed certificate on all target servers (IIS via WinRM, Azure App Service via Azure SDK). Verifies HTTPS is working after each install. Sends a Teams approval card before deploying to Production — and waits for human approval.

Deployment methods:

- IIS servers — PowerShell via WinRM (remote execution)
- Azure App Service — Azure CLI / Azure SDK

Input: Certificate details, new thumbprint, deployment targets **Output:**

```
{
  "deployment_plan": [
    {
      "environment": "Dev",
      "target": "IIS — devserver01",
      "method": "WinRM PowerShell",
      "commands": [
        "Import-PfxCertificate -FilePath cert.pfx -CertStoreLocation Cert:\\LocalMachine\\My",
        "Set-WebBinding -Name 'DefaultWebSite' -BindingInformation '*:443:' -CertificateThumbprint '3A9F...' -Cert
      ],
      "https_verification": "Invoke-WebRequest https://api-dev.example.com -UseBasicParsing",
      "status": "DEPLOYED"
    },
    {
      "environment": "Production",
      "target": "Azure App Service — api-prod",
      "status": "PENDING_APPROVAL"
    }
  ],
  "teams_approval_card": {
    "title": "Certificate Deployment — Production Approval Required",
    "domain": "api.coreandmain.com",
    "new_expiry": "2027-04-27",
    "actions": ["APPROVE", "REJECT"]
  },
  "production_gate": "AWAITING_HUMAN_APPROVAL"
}
```

---

5. Technology Stack

| Layer | Technology | Version | Purpose | |---|---|---|---| | **Agent runtime** | Python | 3.11.9 | All agent logic | | **POC UI** | Streamlit | 1.41.0 | Demo dashboard | | **AI / LLM** | Azure OpenAI (gpt-4o) | API v2024-08-01 | All agent reasoning | | **AI client** | OpenAI Python SDK | 1.57.0 | Azure-compatible client | | **HTTP client** | requests | 2.32.3 | Azure DevOps REST API calls | | **Config** | python-dotenv | 1.0.1 | `.env` file loading | | **Azure Functions** | Python 3.11 | — | Production serverless compute | | **Azure Durable Functions** | — | — | Quality Gate parallel orchestration | | **Azure Service Bus** | — | — | Agent-to-agent messaging | | **Azure Key Vault** | — | — | Secrets + certificate storage | | **Azure Table Storage** | — | — | Certificate inventory | | **Azure Blob Storage** | — | — | Immutable audit logs | | **Azure Monitor** | — | — | Centralized logging and alerts | | **Azure API Management** | — | — | Webhook ingress and auth | | **IaC** | Azure Bicep | — | Infrastructure provisioning | | **Source control** | Git / Azure DevOps | — | Code and PR management | | **Security scanning** | Fortify SSC | REST API | SAST findings source | | **Code under review** | C# / .NET 4.8–10 | — | Quality Gate target | | **Code under review** | TypeScript / Angular | — | Quality Gate target |

---

6. Project Folder Structure

```
blueline/
│
├── poc/                              ← Proof of Concept (runnable)
│   ├── app.py                        ← Main Streamlit dashboard (4 tabs)
│   ├── requirements.txt              ← Python dependencies
│   ├── run.bat                       ← Windows quick-start script
│   ├── .env.example                  ← Config template — copy to .env
│   ├── .gitignore                    ← Excludes .env, __pycache__, images
│   │
│   ├── agents/                       ← All 12 agent implementations
│   │   ├── __init__.py
│   │   ├── clarion.py                ← Coding standards checker
│   │   ├── lumen.py                  ← Code smell detector
│   │   ├── vector.py                 ← Risk and complexity scorer
│   │   ├── ascent.py                 ← Aggregator and final recommendation
│   │   ├── bulwark.py                ← Fortify finding triage
│   │   ├── watchtower.py             ← Fortify scan monitor
│   │   ├── forge.py                  ← Security fix PR generator
│   │   ├── steward.py                ← Immutable audit log writer
│   │   ├── timeline.py               ← Certificate expiry analyser
│   │   ├── regent.py                 ← Certificate inventory manager
│   │   ├── courier.py                ← CA renewal requester
│   │   └── harbour.py                ← Certificate deployment agent
│   │
│   ├── utils/                        ← Shared utilities
│   │   ├── __init__.py
│   │   ├── llm_client.py             ← Azure OpenAI API connector
│   │   ├── azure_devops.py           ← Azure DevOps REST API client
│   │   └── pr_runner.py              ← Quality Gate PR orchestrator
│   │
│   └── samples/                      ← Demo and test files
│       ├── bad_csharp.cs             ← C# with intentional violations
│       ├── bad_typescript.ts         ← TypeScript with intentional violations
│       ├── fortify_finding.txt       ← Sample Fortify finding for security demo
│       └── ado_test_files/           ← 4 realistic files for live PR demo
│           ├── InventoryService.cs   ← BLOCK scenario (critical violations)
│           ├── UserController.cs     ← REQUEST_CHANGES scenario
│           ├── stock-upload.component.ts ← BLOCK scenario (Angular security)
│           ├── order-list.component.ts   ← APPROVE scenario (clean code)
│           └── HOW_TO_USE.md         ← Instructions for demo PR setup
│
├── docs/
│   └── das_review_standards.md       ← DAS/CDAS standards (loaded by CLARION/LUMEN)
│
├── BlueLine_Understanding_Document.md ← Project scope and objectives
├── BlueLine_LLD.md                   ← Full low-level technical design
├── BlueLine_Agent_Architecture_Overview.md ← How agents are built
├── BlueLine_POC_Setup_Guide.md       ← Step-by-step POC setup
├── BlueLine_POC_Test_Cases.md        ← Test scenarios
├── BlueLine_RnD_Research_Document.md ← Technology evaluation and justification
```

```
├── BlueLine_Discovery_Questions.md    ← Discovery Q&A
├── BlueLine_How_Agents_Are_Built.md   ← Accessible agent design explanation
├── BlueLine_Key_Questions_From_Meeting.md ← Meeting Q&A and Spec Kit comparison
├── BlueLine_Team_QnA_Prep.md          ← Stakeholder Q&A prep
├── BlueLine_Demo_Talking_Points.md    ← Demo script
├── BlueLine_Project_Handover_Document.md ← Client handover document
├── AI_Tools_Overview_CodeReview_Security.md ← Market tool comparison
└── BlueLine_Project_Documentation.md ← This document
```

---

7. Local Setup & Installation

Prerequisites

| Requirement | Version | Notes | |---|---|---| | Python | 3.9+ | 3.11.9 confirmed working | | pip | Any | Comes with Python | | Azure OpenAI access | — | Endpoint + API key from Azure admin | | Azure DevOps PAT | — | For live PR review tab only | | Internet access | — | Calls Azure OpenAI and Azure DevOps APIs |

Step 1 — Clone the Repository

```
git clone https://github.com/sagu25/blueline.git
cd blueline
```

Step 2 — Install Dependencies

```
cd poc
pip install -r requirements.txt
```

requirements.txt:

```
openai==1.57.0
streamlit==1.41.0
python-dotenv==1.0.1
requests==2.32.3
```

Step 3 — Create Your `.env` File

Copy the template and fill in your credentials:

```
cp .env.example .env
```

Edit `.env` — see Section 8 for all variables.

Step 4 — Run the Application

Windows:

```
run.bat
```

Any OS:

```
streamlit run app.py
```

The app opens in your browser at `http://localhost:8501`.

---

8. Configuration & Environment Variables

Create `poc/.env` with these values. **Never commit this file — it is in `.gitignore`.**

Azure OpenAI (Required — all agents)

```
AZURE_OPENAI_ENDPOINT=https://YOUR-RESOURCE-NAME.openai.azure.com/
AZURE_OPENAI_API_KEY=your-api-key-here
AZURE_OPENAI_DEPLOYMENT=gpt-4o
```

How to get these:

- Azure Portal → Your OpenAI resource → Keys and Endpoint
- Deployment name: Azure OpenAI Studio → Deployments

Azure DevOps (Required — Live PR Review tab only)

```
AZURE_DEVOPS_ORG_URL=https://dev.azure.com/your-org-name
AZURE_DEVOPS_PAT=your-personal-access-token
AZURE_DEVOPS_PROJECT=YourProjectName
AZURE_DEVOPS_REPO=YourRepositoryName
```

How to get the PAT:

1. Azure DevOps → Profile picture (top right) → Personal access tokens
2. New Token → Name: `BlueLine` → Expiry: 90 days
3. Scopes: `Code → Read` and `Pull Request Threads → Read & Write`
4. Copy immediately — shown only once

How to get Org URL, Project, Repo:

```
https://dev.azure.com/your-org/YourProject/_git/YourRepo
                     ^^^^^^^^  ^^^^^^^^^^^       ^^^^^^^^
                     ORG_URL   PROJECT           REPO
```

---

9. Running the POC Application

The POC is a Streamlit dashboard with four tabs:

Tab 1 — Quality Gate (Code Review)

Purpose: Demonstrate the full CLARION → LUMEN → VECTOR → ASCENT pipeline on a code sample.

How to use:

1. Paste C# or TypeScript code into the text area (or use the preloaded samples)
2. Select language
3. Click **Run Quality Gate**
4. Agents run in sequence, results appear with progress indicators
5. Final ASCENT recommendation shown at bottom

Sample files available:

- `samples/bad_csharp.cs` — multiple CLARION and LUMEN violations
- `samples/bad_typescript.ts` — Angular anti-patterns and security issues

Tab 2 — Security Loop

Purpose: Demonstrate BULWARK → FORGE → STEWARD pipeline on a Fortify finding.

How to use:

1. Paste a Fortify finding description (or use the preloaded sample from `samples/fortify_finding.txt`)
2. Optionally paste the vulnerable code snippet
3. Click **Run Security Analysis**
4. BULWARK classifies, FORGE generates draft PR, STEWARD logs
5. All three outputs shown side by side

Tab 3 — Certificate Loop

Purpose: Demonstrate TIMELINE → REGENT → COURIER → HARBOUR pipeline.

How to use:

1. Select a certificate from the REGENT inventory (4 sample certs loaded)
2. Click **Run Certificate Analysis**
3. TIMELINE assesses urgency, COURIER simulates renewal, HARBOUR generates deployment plan
4. Teams approval card preview shown for Production deployment

---

Tab 4 — Live PR Review

Purpose: Run the Quality Gate against a **real Pull Request** from your Azure DevOps project.

Requires: Azure DevOps credentials in `.env` (see Section 8).

How to use:

1. Enter your ADO Org URL, Project, Repo, and PAT in the sidebar (or set in `.env`)
2. Click **Load PRs** — live PRs from your repo appear
3. Select a PR from the dropdown
4. Toggle **Shadow Mode** (on = agents run but post no comments; off = comments posted to real PR)
5. Click **Run Live Review**
6. Agents fetch real PR diff, review each .cs and .ts file, and aggregate
7. If shadow mode is off — inline comments and summary comment posted directly to the PR in ADO

---

10. System Integrations

Azure DevOps Integration

File: `poc/utils/azure_devops.py` **API version:** 7.1 **Authentication:** PAT via Basic auth (base64 encoded)

Available operations:

| Method | Description | |---|---| | `list_pull_requests(status, top)` | Fetch open/active PRs from the configured repo | | `get_pr_details(pr_id)` | Get PR title, source branch, target branch, creator | | `get_pr_changed_files(pr_id)` | List changed `.cs` and `.ts` files only | | `get_file_content(path, branch)` | Fetch raw file content from a specific branch | | `post_inline_comment(pr_id, path, line, content)` | Post review comment on a specific line | | `post_pr_summary(pr_id, content)` | Post ASCENT summary comment on the PR |

API Endpoints used:

```
GET  /{org}/{project}/_apis/git/repositories/{repo}/pullrequests
GET  /{org}/{project}/_apis/git/repositories/{repo}/pullrequests/{id}
GET  /{org}/{project}/_apis/git/repositories/{repo}/pullrequests/{id}/iterations/{i}/changes
GET  /{org}/{project}/_apis/git/repositories/{repo}/items?path={}&versionDescriptor.version={}
POST /{org}/{project}/_apis/git/repositories/{repo}/pullrequests/{id}/threads
```

Azure OpenAI Integration

File: `poc/utils/llm_client.py`

How it works:

```
# Every agent call follows this pattern:
response = call_claude(
    system_prompt="You are CLARION...",   # agent identity + rules
    user_message="Review this code: ...", # the actual input
    max_tokens=4096,
    temperature=0.1                        # low temperature for consistency
)
```

Configuration check: The `get_active_provider()` function checks if Azure OpenAI env vars are set and returns the configured provider. If not configured, the UI shows a warning.

Fortify SSC Integration (Production)

Agent: WATCHTOWER **Protocol:** REST API **Authentication:** API token

Operations needed in production:

- `GET /api/v1/projectVersions/{id}/issues` — fetch new findings
- `GET /api/v1/issues/{id}` — get finding detail
- `POST /api/v1/issues/action/suppress` — suppress false positives

Access requirement: Fortify SSC API URL + token with read access to your project — must be stored in Azure Key Vault as `fortify-api-token`.

Azure Key Vault Integration (Production)

Agents: TIMELINE (read), REGENT (read/write), COURIER (write), HARBOUR (read)

Operations:

- List certificates and their metadata

- Get certificate expiry dates
- Store renewed PFX files
- Retrieve certificates for deployment

Access requirement: BlueLine Managed Identity needs `Certificate Get`, `Certificate List`, `Certificate Import` permissions on the Key Vault.

IIS / WinRM Integration (Production)

Agent: HARBOUR **Protocol:** WinRM (Windows Remote Management) via PowerShell

PowerShell commands generated by HARBOUR:

```
# Import certificate to local machine store
Import-PfxCertificate -FilePath "C:\certs\api.pfx" `
  -CertStoreLocation Cert:\LocalMachine\My `
  -Password (ConvertTo-SecureString "pass" -AsPlainText -Force)

# Bind to IIS site
Set-WebBinding -Name "DefaultWebSite" `
  -BindingInformation "*:443:" `
  -CertificateThumbprint "3A9F..." `
  -CertificateStoreName "My"

# Verify HTTPS
Invoke-WebRequest https://api.example.com -UseBasicParsing
```

Access requirement: WinRM enabled on target servers; service account with `Import PFX` and IIS binding rights.

Microsoft Teams Integration (Production)

Agent: HARBOUR (Production approval gate) **Protocol:** Incoming webhook

What is sent: An Adaptive Card with certificate details and two buttons — **APPROVE** and **REJECT** — posted to a designated Teams channel. HARBOUR waits for the approval response before proceeding with Production deployment.

---

11. Production Architecture

Azure Resource Groups

```
blueline-rg-shared     ← Key Vault, Service Bus, Storage, Log Analytics, API Management
blueline-rg-quality    ← Quality Gate Function App
blueline-rg-security   ← Security Loop Function App
blueline-rg-certloop   ← Certificate Loop Function App
```

Azure Function Apps

| Function App | Plan | Always-On | Trigger | |---|---|---|---| | `blueline-quality-fa` | Premium EP1 | Yes | HTTP (PR webhook — needs fast response) | | `blueline-security-fa` | Consumption | No | HTTP (pipeline hook) + Timer | | `blueline-certloop-fa` | Consumption | No | Timer (daily at 06:00 UTC) |

> *Quality Gate uses Premium plan so Function App is always warm — cold start would delay PR review comments by 30–90 seconds.*

Shared Resources

| Resource | Type | Purpose | |---|---|---| | `blueline-kv` | Azure Key Vault | API keys, CA credentials, certificate files | | `blueline-sb` | Azure Service Bus | Agent-to-agent messaging (topics + subscriptions) | | `blueline-storage` | Azure Storage Account | STEWARD audit logs (immutable blobs) | | `blueline-tables` | Azure Table Storage | REGENT certificate inventory | | `blueline-law` | Log Analytics Workspace | All agent logs, dashboards, alerts | | `blueline-apim` | Azure API Management | Webhook ingress, auth, rate limiting |

Azure Service Bus Topics

| Topic | Publisher | Subscriber | |---|---|---| | `security.findings.new` | WATCHTOWER | BULWARK | | `security.critical.fix-needed` | BULWARK | FORGE | | `security.all-events` | All security agents | STEWARD | | `certificate.renewal.requested` | TIMELINE | COURIER | | `certificate.deployed` | HARBOUR | STEWARD |

Infrastructure as Code

Infrastructure is provisioned using **Azure Bicep** templates in the `infrastructure/` folder. Deploy with:

```
az deployment group create \
  --resource-group blueline-rg-shared \
  --template-file infrastructure/main.bicep \
  --parameters @infrastructure/parameters.json
```

---

12. Human Control Points & Safety Design

BlueLine is built on the principle that **no critical action happens automatically without a human approving it first.**

Approval Gates

| Action | Automatic? | Approval method | |---|---|---| | Posting inline review comments on PR | Yes | No approval needed — informational | | Merging an agent-reviewed PR | No | Standard ADO PR approval by human reviewer | | Creating a FORGE security fix branch | Yes | No approval needed — just creates the branch | | Merging a FORGE-generated fix PR | No | Human reviews draft PR and approves in ADO | | Deploying certificate to Dev | Yes | No approval needed | | Deploying certificate to QA | Yes | No approval needed | | Deploying certificate to Production | No | Human clicks Approve on Teams card | | Suppressing a Fortify finding | Agent recommends | InfoSec reviews STEWARD log periodically |

Confidence Threshold

Every agent output includes a confidence score (0.0–1.0). If confidence is below **0.7**, the agent:

1. Does **not** take the action
2. Logs the reason
3. Sends a notification to the configured Teams channel
4. The item remains in the queue for human review

Escalation Method (`escalate()`)

Every agent inherits an `escalate()` method from `BaseAgent`. When called:

- Action is halted immediately
- Reason is logged to STEWARD
- Notification sent via Teams
- Run ID included so the human can find the full context

Destructive Actions — Never Automated

Agents are explicitly instructed in their system prompts to **never**:

- Delete or revoke a certificate
- Merge code (agents only create draft PRs)
- Suppress a finding without logging a reason
- Deploy to Production without human approval
- Discard a message — failed messages go to a dead-letter queue for review

---

13. Shadow Mode

Shadow mode is a critical safety feature for initial rollout.

When shadow mode is ON:

- Agents run the full analysis pipeline
- All output is generated and logged
- **No external actions are taken** — no PR comments posted, no Service Bus messages published, no deployments triggered
- Engineering lead can review logs and validate output quality

When shadow mode is OFF:

- Agents take real actions — post comments, create PRs, deploy certs
- All actions are logged to STEWARD

Recommended rollout:

1. **Sprint 1–2:** Shadow mode ON for Quality Gate — review all output in logs, tune false positives
2. **Sprint 3:** Shadow mode OFF for Quality Gate — agents start posting live PR comments
3. **Sprint 4–5:** Shadow mode ON for Security Loop — validate BULWARK classification accuracy
4. **Sprint 6:** Shadow mode OFF for Security Loop
5. **Month 2:** Certificate Loop in read-only mode — validate TIMELINE expiry detection
6. **Month 3:** Certificate Loop fully live with human Prod gate active

In the POC: Shadow mode toggle is on the Live PR Review tab (Tab 4) in the Streamlit UI.

---

14. Testing Guide

Test Cases — Quality Gate

| Scenario | Input File | Expected Outcome | |---|---|---| | Critical C# violations | `samples/ado_test_files/InventoryService.cs` | ASCENT → BLOCK; CLARION flags .Result, hardcoded secret | | Medium violations | `samples/ado_test_files/UserController.cs` | ASCENT → REQUEST_CHANGES; LUMEN flags long method | | Angular security issues | `samples/ado_test_files/stock-upload.component.ts` | ASCENT → BLOCK; CLARION flags unsafe HTML binding | | Clean code | `samples/ado_test_files/order-list.component.ts` | ASCENT → APPROVE; minor suggestions only |

Test Cases — Security Loop

| Scenario | Input | Expected Outcome | |---|---|---| | SQL Injection finding | Paste from `samples/fortify_finding.txt` | BULWARK → CRITICAL; FORGE generates parameterized query fix PR | | False

positive finding | Low-severity informational finding | BULWARK → FALSE_POSITIVE with reasoning | | Ambiguous finding | Complex pattern with context dependency | BULWARK → NEEDS_REVIEW; escalates to human |

Test Cases — Certificate Loop

| Scenario | Certificate | Expected Outcome | |---|---|---| | Expiring in 5 days | REGENT sample cert 1 | TIMELINE → CRITICAL; COURIER → renew immediately; HARBOUR → deploy + Prod gate | | Expiring in 20 days | REGENT sample cert 2 | TIMELINE → URGENT; action plan generated | | Healthy cert | REGENT sample cert 4 | TIMELINE → OK; no action required |

Running the Full POC Test

1. Set up `.env` with Azure OpenAI credentials
2. Open each tab in sequence
3. Use the sample files and preloaded inputs
4. Verify each agent produces output matching the expected outcome above
5. For Tab 4 (Live PR Review), push the `ado_test_files/` to a branch and create a real PR

---

15. Deployment Guide

Production Deployment Steps

Step 1 — Provision Infrastructure

```
# Deploy shared resources first
az deployment group create \
  --resource-group blueline-rg-shared \
  --template-file infrastructure/shared.bicep

# Deploy each track
az deployment group create \
  --resource-group blueline-rg-quality \
  --template-file infrastructure/quality-gate.bicep

az deployment group create \
  --resource-group blueline-rg-security \
  --template-file infrastructure/security-loop.bicep

az deployment group create \
  --resource-group blueline-rg-certloop \
  --template-file infrastructure/cert-loop.bicep
```

Step 2 — Store Secrets in Key Vault

```
az keyvault secret set --vault-name blueline-kv --name "anthropic-api-key" --value "YOUR_KEY"
az keyvault secret set --vault-name blueline-kv --name "ado-pat" --value "YOUR_PAT"
az keyvault secret set --vault-name blueline-kv --name "fortify-api-token" --value "YOUR_TOKEN"
az keyvault secret set --vault-name blueline-kv --name "ca-api-key" --value "YOUR_CA_KEY"
az keyvault secret set --vault-name blueline-kv --name "teams-webhook-url" --value "YOUR_WEBHOOK"
```

Step 3 — Configure ADO Webhook

In Azure DevOps → Project Settings → Service Hooks → Create subscription:

- Trigger: `Pull request created` and `Pull request updated`
- URL: `https://blueline-apim.azure-api.net/webhook/pr`
- Authentication: API key from Azure API Management

Step 4 — Load Coding Standards

Copy `docs/das_review_standards.md` content into the CLARION and LUMEN Function App settings as the `DAS_STANDARDS_CONTENT` environment variable. When standards change, update this variable and restart the Function App — no code deployment needed.

Step 5 — Enable Shadow Mode (Initial Rollout)

Set `SHADOW_MODE=true` in all Function App configuration settings. Remove after validation (see Section 13).

Step 6 — Configure Azure Monitor Alerts

Set up alerts in Log Analytics for:

- Any agent function failure → Teams notification
- Quality Gate not responding to webhooks > 5 minutes → DevOps alert
- Certificate expiry < 7 days detected → Ops + InfoSec alert
- BULWARK false positive rate > 20% → Engineering lead weekly report

---

16. DAS Coding Standards Reference

The file `docs/das_review_standards.md` is the single source of truth for coding standards loaded into CLARION and LUMEN. It covers:

C# / .NET Standards:

- Naming conventions (classes, interfaces, methods, variables, constants)
- Async/await patterns and threading rules
- Dependency injection requirements
- Exception handling patterns

- Security requirements (no hardcoded secrets, input validation, parameterized queries)
- Entity Framework usage (AsNoTracking, migration conventions)
- Logging patterns (structured logging, log levels, what to log)
- CORS configuration rules
- API response standards (consistent error format, status codes)

TypeScript / Angular Standards:

- Component naming and file naming
- Change detection strategy (OnPush required)
- HTTP client usage (typed services, no direct HttpClient in components)
- State management patterns
- Template security (no innerHTML, no bypassSecurityTrust without review)
- Observable usage and subscription management
- Module organization

To update standards: Edit `docs/das_review_standards.md` and update the Function App `DAS_STANDARDS_CONTENT` environment variable. No code deployment required.

---

17. Glossary

| Term | Definition | |---|---| | **Agent** | An autonomous AI-powered software component responsible for one discrete task in a track | | **Track** | A named automation pipeline for one of the three problem areas | | **ASCENT** | Aggregates CLARION + LUMEN + VECTOR output into one PR recommendation | | **BULWARK** | Triages Fortify security findings into severity classifications | | **CLARION** | Checks code against .NET and Angular coding standards | | **COURIER** | Requests certificate renewal from a Certificate Authority | | **FORGE** | Generates a code fix and creates a draft PR for security vulnerabilities | | **HARBOUR** | Deploys renewed certificates to IIS servers and Azure App Services | | **LUMEN** | Detects code smells and anti-patterns | | **REGENT** | Maintains the structured certificate inventory | | **STEWARD** | Writes immutable audit log entries for all security decisions | | **TIMELINE** | Monitors certificate expiry dates and flags renewals needed | | **VECTOR** | Scores code risk and complexity; identifies hotspots for reviewer focus | | **WATCHTOWER** | Monitors Fortify SSC for new findings on a schedule | | **Shadow Mode** | Operating mode where agents analyse but take no real actions — for validation | | **Human Gate** | A mandatory pause requiring explicit human approval before the pipeline continues | | **Confidence Score** | A 0.0–1.0 score; below 0.7 the agent escalates to a human instead of acting | | **Fortify SSC** | OpenText Fortify Software Security Center — the SAST tool integrated in the Security Loop | | **CA** | Certificate Authority — entity that issues SSL/TLS certificates (DigiCert, internal PKI) | | **IIS** | Internet Information Services — Microsoft web server used for .NET hosting and cert binding | | **WinRM** | Windows Remote Management — used by HARBOUR for remote PowerShell execution on IIS servers | | **DAS / CDAS** | Design and Architecture Standards / Coding and Development Architecture Standards — the customer's coding rulebook | | **LLM** | Large Language Model — the AI model (Claude via Azure OpenAI) used by all agents | | **PR** | Pull Request — a code

change submitted for review in Azure DevOps | | **WORM** | Write Once Read Many — Azure Blob Storage policy used for immutable audit logs | | **MTTR** | Mean Time to Remediate — key metric measuring how fast security findings are fixed |

Project BlueLine — Complete Project Documentation v1.0 LTM AI-Led Engineering Team | April 2026 Repository: https://github.com/sagu25/blueline.git